

Flutter Fundamentals

Short recap day #1



Peter Kassenaar –
info@kassenaar.com

Day #1



- **Introduction** – overview of the Flutter landscape
- Flutter **tooling** – installation
- Hello World –the **structure and architecture** of Flutter apps.
- Zooming in on Flutter:
 - Components – **Stateless** and **Stateful**

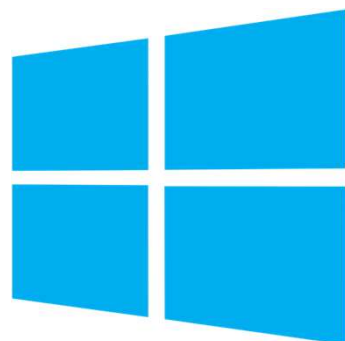


Day #1, cont'd

- Widgets - introduction
- The Flutter layout system – `Scaffold()` and more
- Using Images and assets
- More layout widgets
 - `Button()`, `Icon()`, `Container()`, `Padding()`
 - `Row()`, `Column()`, properties, children and more
- Optional: working with data
 - `ListView()`, `Card()`, Designing layouts



*"Flutter is **Google's UI toolkit** for building beautiful, **natively compiled** applications for mobile, web, and desktop from a **single codebase.**"*



Installation – recommended order



1. Install Visual Studio
2. Install IntelliJ
3. Install Flutter plug-in for IntelliJ
4. Install Flutter SDK
5. Update Windows PATH variable
6. Run `flutter doctor`, fix any possible problems

Default code – recognize the structure



```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        visualDensity: VisualDensity.adaptivePlatformDensity,
      ),
      home: MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}
...
```

(Yours might be slightly different,
due to updates)

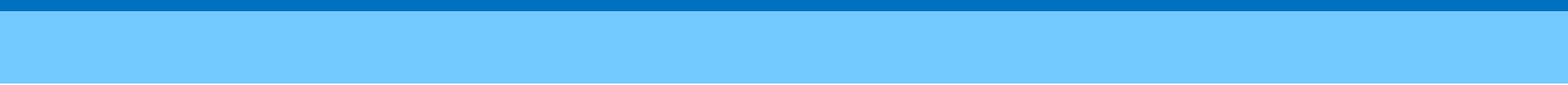
Read the [useful comments](#).

Flutter == Dart in action



- Important widgets
 - `Scaffold()`
 - `AppBar()`
 - Themes, fonts & colors
 - `Image()`
 - `Icon()`
 - `Row()`, `Column()`
 - `ListView()`
 - `Container()`
 - `Expanded()`

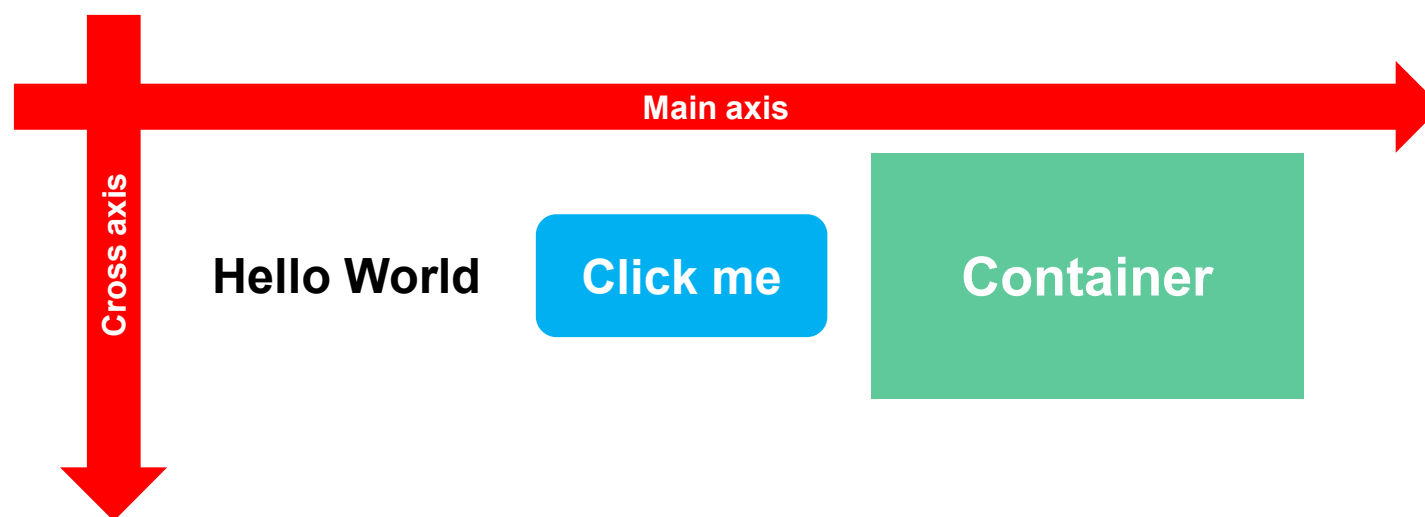
*“Every Flutter App
is composed as a
tree of widgets”*





Alignment in Rows/Columns

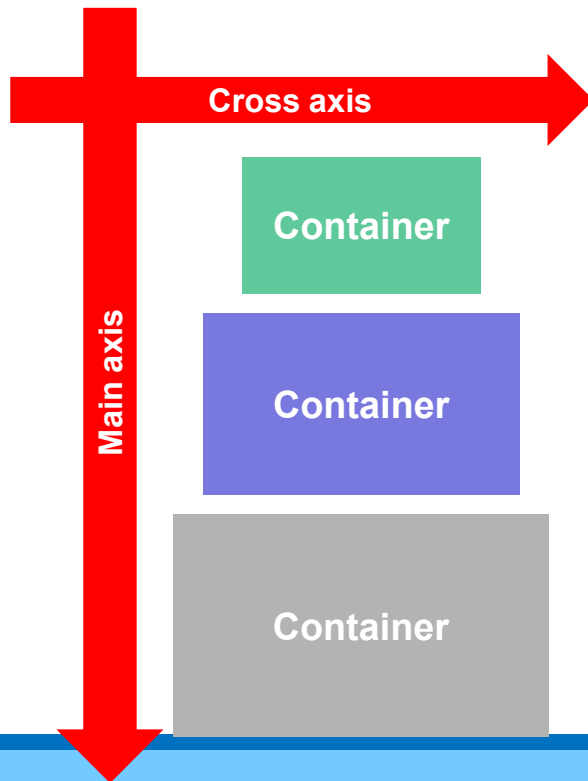
- In Rows:
 - Use `MainAxisAlignment` for horizontal layout
 - Use `CrossAxisAlignment` for vertical layout



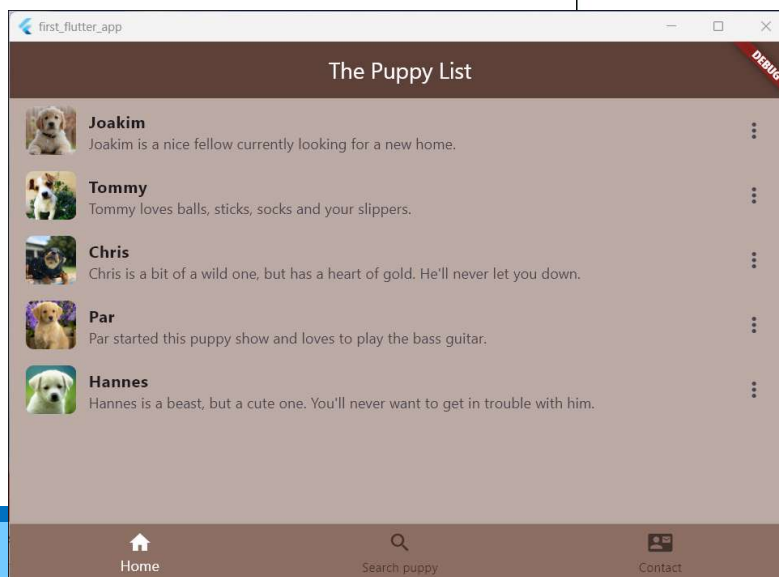
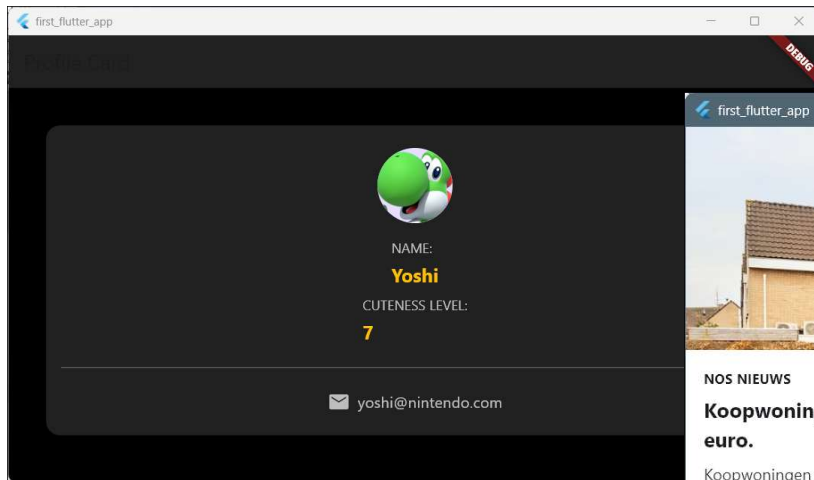
Alignment



- In Columns – opposite to Rows:
 - Use `MainAxisAlignment` for vertical layout
 - Use `CrossAxisAlignment` for horizontal layout



Creating various layouts by combining Widgets



Creating Layouts



Design rows and columns. Start global, then drill down

Row
2 children

child: new Column child: new Image

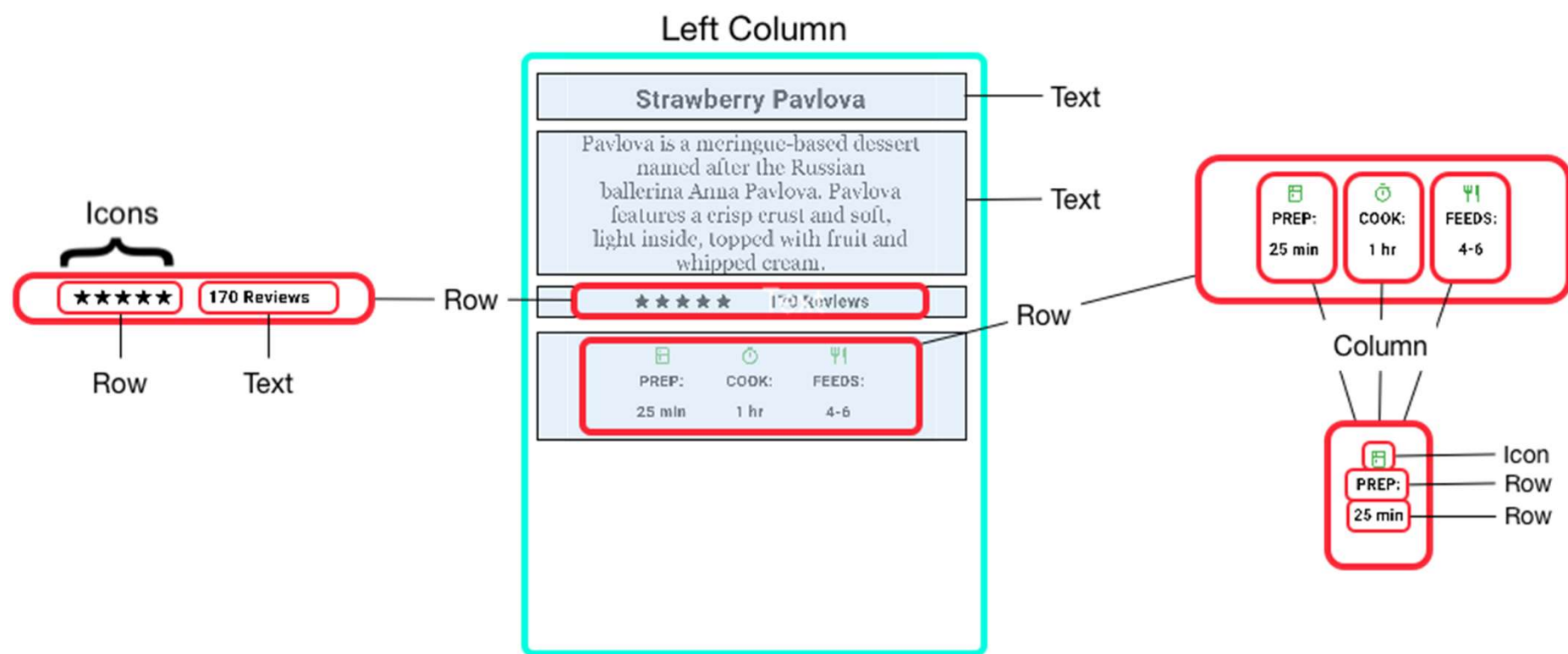
Column
4 children

Strawberry Pavlova

Pavlova is a meringue-based dessert named after the Russian ballerine Anna Pavlova. Pavlova features a crisp crust and soft, light inside, topped with fruit and whipped cream.

★★★★★ 170 Reviews

PREP:	COOK:	FEEDS:
25 min	1 hr	4-6

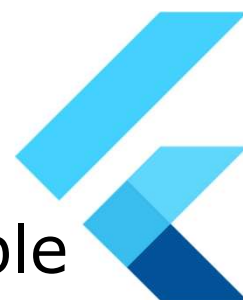




Questions:

- Using `const` or not in code?
- Use `const` in situations where the following is true:
 - The object is **entirely immutable**.
 - Its **parameters** (if any) are *also compile-time constants*.
 - You want to **avoid recreating the object unnecessarily** every time a widget rebuilds.
- E.g: consider this snippet:

```
@override  
Widget build(BuildContext context) {  
    return const Placeholder();  
}
```



- Removing the keyword `const` is less optimal!
- It still works because `Placeholder()` is immutable by design.
- However, removing `const` makes a difference in performance!
 - Without `const`, a *new instance* of `Placeholder()` will be created every time this widget rebuilds.
 - With `const`, the *same compile-time constant instance is reused* across rebuilds, improving performance by avoiding unnecessary object creation!

```
Widget build(BuildContext context) {  
    return Placeholder(); // also works!  
}
```

verdict



- Use `const` when the value is completely fixed and known at compile-time.
 - This could include compile-time constants or immutable values like widget configurations.
 - Performance optimization of apps.
- Use `final` when the value cannot change after it is assigned, but its assignment might depend on runtime conditions or calculations.

Function `build()` called at 60fps?



- In the following example, the function `print()` is NOT called 60 times per second. Why?

```
class Home extends StatelessWidget {  
  Widget build(BuildContext context) {  
    print('function build called');  
    return Scaffold(  
      appBar: AppBar()  
      ...  
    )  
  }  
}
```



Answer:

“Flutter's 60fps refers to the *painting/rendering* rate, not the *build* rate. These are two different phases.”

- The `build()` method in this widget is only called when:
 - **Initial creation:** When the widget is *first created and added* to the widget tree (and yes, *then* we see the message printed in the console)
 - **Parent rebuild:** When the parent widget *rebuilds and creates a new instance* of `Home`
 - **State changes:** If this were a `StatefulWidget` and *`setState()` was called*

From the docs:



*"This is actually one of Flutter's key performance optimizations - **separating the logical structure (build) from the visual rendering (paint).**"*



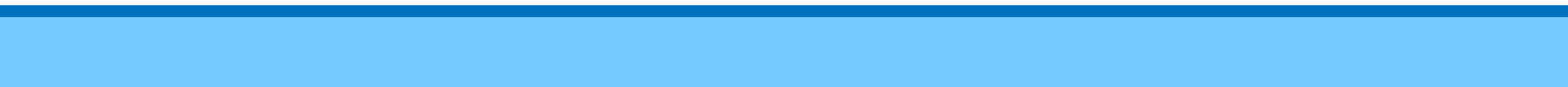
other

Questions?

Today:



- State management in various ways
 - Stateful widgets – using models/custom classes
 - passing parameters, passing functions
- Communicating with external API's
 - http / other methods
- More state management
 - bloc pattern, bloc + cubit implementation



Tomorrow



- Wrapping up state management
- TextFields
- Routing / Navigation
- Complete applications
- gRPC
- Gestures
- Publication (executables/packages)
- Evals & goodbye
- ...